Python Programming - Module 1

Theory Questions with Answers

Answers are written in short exam-ready form. Learn the points, then rewrite in your own words. Questions are grouped sub-module wise (1.1 to 1.4). Marks suggestion: Section A = 2 marks, Section B = 4-5 marks.

Module 1.1 - Python Basics

Section A - Short Answer (2 marks)

Q1. What is indentation in Python? Why is it important?

Ans: Indentation is the leading whitespace at the start of a line. Python uses it to define blocks (if, loops, functions, classes) instead of braces. Standard is 4 spaces. Inconsistent indentation raises IndentationError.

Q2. How do you write single-line and multi-line comments?

Ans: Single-line comments start with #. Python has no true multi-line comment; triple-quoted strings (''' ... ''' or """ ... """) are used as multi-line comments/docstrings.

Q3. What does input() return? How do you read an integer?

Ans: input() always returns a string. Use int(input()) to read an integer and float(input()) for a real number.

Q4. Explain the sep and end parameters of print().

Ans: sep is the separator placed between multiple values (default is a space). end is the string printed at the end (default is newline '\n').

```
print('A','B',sep='-',end='!')   # A-B!
```

Q5. Differentiate between implicit and explicit type conversion.

Ans: Implicit: Python converts automatically (e.g. 5 + 2.0 gives 7.0). Explicit (type casting): programmer converts using int(), float(), str(), etc. (e.g. int('10')).

Q6. What is the difference between == and is?

Ans: == compares the values of two objects; is compares their identity (whether both refer to the same object in memory, same id()).

Q7. What is the use of the type() function?

Ans: type(x) returns the data type (class) of the object x, e.g. type(5) gives <class 'int'>.

Q8. What is meant by 'Python is dynamically typed'?

Ans: Variable types need not be declared; the type is determined at run time from the assigned value, and the same variable can later hold a different type.

Q9. What is meant by 'Python is strongly typed'?

Ans: Python does not implicitly convert incompatible types. For example '5' + 2 raises TypeError; explicit conversion is required.

Q10. What are identifiers? State the rules for naming them.

Ans: Identifiers are names of variables, functions, classes, etc. Rules: letters, digits and underscore only; cannot start with a digit; no spaces or special symbols; cannot be a keyword; case-sensitive.

Q11. What are keywords? Give five examples.

Ans: Keywords are reserved words with predefined meaning that cannot be used as identifiers. Examples: if, else, for, while, def, class, True, None, import, return.

Q12. Differentiate between a statement and an expression.

Ans: An expression evaluates to a value (e.g. 2 + 3, x > 5). A statement is a complete instruction that performs an action (e.g. x = 5, print(x), if ...). Expressions can appear inside statements.

Q13. What is the difference between / and // operators?

Ans: / is true division and always returns a float (7/2 = 3.5). // is floor division and returns the largest integer not greater than the result (7//2 = 3, -7//2 = -4).

Q14. What is the value of 2 ** 3 ** 2? Why?

Ans: 512. The ** operator is right-associative, so it is evaluated as 2 ** (3 ** 2) = 2 ** 9 = 512.

Q15. List the logical operators in Python.

Ans: and, or, not. They use short-circuit evaluation: in 'a and b', b is evaluated only if a is True.

Section B - Descriptive (4-5 marks)

Q16. Explain operator precedence and associativity with a table and examples.

Ans: Precedence decides which operator is evaluated first; associativity decides order for operators of the same precedence. Order (high to low): ( ) > ** > unary +,-,~ > *,/,//,% > +,- > <<,>> > & > ^ > | > comparisons, is, in > not > and > or. All binary operators are left-to-right except **, which is right-to-left.

```
print(10 - 4 - 2)      # 4   (left to right)
print(2 + 3 * 4 ** 2)  # 50
print(not 0 or 0)      # True
```

Q17. Explain the different types of operators in Python with examples.

Ans: Arithmetic (+ - * / // % **), Relational (== != > < >= <=), Logical (and or not), Assignment (= += -= etc.), Bitwise (& | ^ ~ << >>), Membership (in, not in) and Identity (is, is not). Example: 7 % 3 = 1, 5 & 3 = 1, 'a' in 'cat' is True, x is y checks same object.

Q18. Explain the built-in data types of Python.

Ans: Numeric: int, float, complex. Boolean: bool (True/False). Sequence: str, list, tuple. Set types: set, frozenset. Mapping: dict. None type: None. Mutable types: list, set, dict. Immutable types: int, float, bool, str, tuple, frozenset.

Q19. Explain type conversion functions with examples.

Ans: int(), float(), str(), bool(), list(), tuple(), set(), dict() convert values between types. int(7.9) = 7 (truncation), float('3') = 3.0, str(10) = '10', list('ab') = ['a','b'], bool('') = False. int('3.5') raises ValueError.

Q20. Differentiate between dynamically typed and strongly typed languages with examples.

Ans: Dynamic typing: type checked at run time, no declarations; x = 10 then x = 'ten' is allowed. Strong typing: no silent conversion between unrelated types; '5' + 2 raises TypeError while '5' + str(2) works. Python is both dynamically and strongly typed.

Q21. Explain variables, assignment and multiple assignment in Python.

Ans: A variable is a name that refers to an object. Assignment uses =. Multiple assignment: a, b = 1, 2; chained: a = b = 0; swapping: a, b = b, a; augmented: x += 1. Python variables are references, so y = x makes both names point to the same object.

Q22. Write a short note on indentation, comments and input/output functions.

Ans: Indentation defines blocks. Comments (#, triple quotes) are ignored by Python and used for documentation. input() reads text from keyboard; print() sends output to screen with sep and end options. Typical use: n = int(input('Enter: ')); print('Square =', n*n).

Module 1.2 - Strings and Lists

Section A - Short Answer (2 marks)

Q23. How can strings be created in Python?

Ans: Using single quotes, double quotes, or triple quotes (for multi-line strings). Raw strings use r'' prefix to ignore escape sequences.

Q24. Why are strings called immutable?

Ans: Once created, characters of a string cannot be changed. s[0] = 'a' raises TypeError. Any modification creates a new string.

Q25. What is the difference between positive and negative indexing?

Ans: Positive index starts from 0 at the left end; negative index starts from -1 at the right end. For 'PYTHON', s[0] = 'P' and s[-1] = 'N'.

Q26. Explain string slicing syntax. What does s[::-1] do?

Ans: s[start:stop:step]; start included, stop excluded. s[::-1] returns the string reversed.

Q27. What is the difference between find() and index()?

Ans: Both return the first index of a substring. If not found, find() returns -1 whereas index() raises ValueError.

Q28. What is the difference between split() and join()?

Ans: split() breaks a string into a list using a separator; join() combines items of an iterable into a single string using a separator string.

Q29. Name any three ways of formatting strings.

Ans: % operator ('%d' % 5), str.format() ('{}'.format(5)) and f-strings (f'{x}').

Q30. What is the difference between append() and extend() in lists?

Ans: append(x) adds x as ONE element at the end (a list stays nested). extend(iterable) adds each element of the iterable individually.

Q31. Differentiate between remove(), pop() and the del statement for lists.

Ans: remove(x) deletes first occurrence by VALUE; pop(i) deletes by INDEX and returns the item (last item by default); del L[i] or del L[i:j] deletes by index/slice and returns nothing; del L deletes the whole list.

Q32. What is the difference between sort() and sorted()?

Ans: list.sort() sorts the list in place and returns None. sorted(x) returns a new sorted list and leaves the original unchanged.

Q33. Name any four built-in functions that can be used on lists.

Ans: len(), max(), min(), sum(), sorted(), any(), all(), list(), enumerate().

Q34. What is the output of [1, 2] * 2 and [1, 2] + [3]?

Ans: [1, 2, 1, 2] and [1, 2, 3].

Q35. What is the use of the insert() method? Give the syntax.

Ans: insert(index, element) inserts the element before the given index, shifting the rest to the right. L.insert(1, 'x').

Q36. Is slicing out of range an error for lists/strings? Explain.

Ans: No. Slicing silently adjusts to the available range (e.g. 'abc'[1:100] = 'bc'). Only direct indexing out of range raises IndexError.

Section B - Descriptive (4-5 marks)

Q37. Explain any eight string methods with examples.

Ans: upper(), lower(), title(), strip(), replace(old,new), split(sep), count(sub), startswith(sub), find(sub), isdigit(), center(w). Methods return new strings.

```
s = ' hello World '
print(s.strip().upper())       # 'HELLO WORLD'
print(s.replace('l','L'))      # ' heLLo WorLd '
print('a,b,c'.split(','))      # ['a','b','c']
print('banana'.count('a'))     # 3
```

Q38. Explain string slicing with examples (including negative step).

Ans: Slice = s[start:stop:step]. Defaults start=0, stop=len(s), step=1. Negative step moves right-to-left.

```
s = 'PYTHON'
print(s[1:4])   # YTH
print(s[:2])    # PY
print(s[-3:])   # HON
print(s[::2])   # PTO
print(s[::-1])  # NOHTYP
```

Q39. Explain different methods of string formatting with examples.

Ans: 1) % formatting uses %d %f %s placeholders. 2) str.format() uses {} with optional index/format spec. 3) f-strings embed expressions directly.

```
n, m = 'Ravi', 88.5
print('%s got %.1f' % (n, m))
print('{} got {:.1f}'.format(n, m))
print(f'{n} got {m:.1f}')
```

Q40. What is a list? Explain how to create a list and perform basic operations.

Ans: A list is an ordered, mutable collection of items in [ ]. Creation: [], [1,2,3], list('abc'), list(range(5)). Operations: concatenation (+), repetition (*), membership (in), len(), iteration with for, indexing and slicing, and item assignment.

Q41. Explain list methods with suitable examples.

Ans: append, extend, insert, remove, pop, clear, index, count, sort, reverse, copy.

```
L = [3, 1, 2]
L.append(5); L.insert(0, 9)   # [9, 3, 1, 2, 5]
L.remove(1); L.pop()          # [9, 3, 2]
L.sort(); print(L)            # [2, 3, 9]
L.reverse(); print(L)         # [9, 3, 2]
```

Q42. Explain indexing and slicing in lists. How can slices be used to modify a list?

Ans: Indexing accesses items by position (0 or -1 based). Slicing L[a:b:c] gives a sub-list. Since lists are mutable, slice assignment replaces part of the list: L[1:3] = [7, 8, 9] replaces two items by three.

Q43. Explain the del statement with respect to lists with examples.

Ans: del removes an item, a slice or the whole list. del L[2] removes index 2; del L[1:3] removes a range; del L deletes the variable so later use gives NameError.

Q44. Differentiate between strings and lists.

Ans: String: sequence of characters only, immutable, written in quotes. List: sequence of any objects, mutable, written in [ ]. Both support indexing, slicing, +, *, in and len().

Module 1.3 - Dictionaries

Section A - Short Answer (2 marks)

Q45. What is a dictionary? How is it created?

Ans: A dictionary stores data as key:value pairs inside { }. Created using {} literal, dict() constructor, or dict(zip(keys, values)).

Q46. What are the rules for dictionary keys?

Ans: Keys must be unique and immutable/hashable (int, float, str, tuple). Lists, sets and dicts cannot be keys. Values can be of any type and can repeat.

Q47. What is the difference between d[key] and d.get(key)?

Ans: d[key] raises KeyError if key is missing; d.get(key) returns None (or a given default) instead.

Q48. What do keys(), values() and items() return?

Ans: keys() gives a view of all keys, values() a view of all values, items() a view of (key, value) tuples.

Q49. What is the use of the update() method?

Ans: update(other) adds key:value pairs from another dictionary (or iterable of pairs) and overwrites values of existing keys.

Q50. Differentiate between pop() and popitem() in a dictionary.

Ans: pop(key) removes that key and returns its value. popitem() removes and returns the last inserted (key, value) pair.

Q51. What does setdefault() do?

Ans: setdefault(k, v) returns d[k] if k exists; otherwise inserts k with value v (default None) and returns v.

Q52. How does the 'in' operator work with dictionaries?

Ans: It checks whether a KEY is present: 'a' in d. To check a value use 'x' in d.values().

Q53. Name four built-in functions that can be used on dictionaries.

Ans: len(), sorted(), min(), max(), sum(), any(), all(), list(), dict(). They operate on keys.

Section B - Descriptive (4-5 marks)

Q54. Explain how to access, add, modify and delete items in a dictionary with examples.

Ans: Access with d[key] or get(); add/modify with d[key] = value; delete with del d[key], pop(), popitem() or clear().

```
d = {'id': 1, 'name': 'Asha'}
d['age'] = 20          # add
d['name'] = 'Isha'     # modify
del d['id']            # delete
print(d)               # {'name': 'Isha', 'age': 20}
```

Q55. Explain any eight dictionary methods with examples.

Ans: keys, values, items, get, update, pop, popitem, setdefault, copy, clear, fromkeys.

```
d = {'a': 1, 'b': 2}
print(d.get('z', 0))        # 0
d.update({'c': 3})
print(list(d.items()))      # [('a',1),('b',2),('c',3)]
print(d.pop('a'))           # 1
print(d.setdefault('k', 9)) # 9
```

Q56. Explain the del statement for dictionaries. How is it different from clear()?

Ans: del d[key] removes one pair; del d removes the dictionary variable itself (later use causes NameError). d.clear() empties the dictionary but the variable still exists as {}.

Q57. Differentiate between a list and a dictionary.

Ans: List: ordered by position, accessed by integer index, items only. Dictionary: accessed by unique key, stores key:value pairs, fast lookup by key, keys must be hashable. Both are mutable.

Q58. How can you iterate through a dictionary? Explain with examples.

Ans: Loop over keys (default), values(), or items().

```
d = {'a': 1, 'b': 2}
for k in d: print(k)
for v in d.values(): print(v)
for k, v in d.items(): print(k, v)
```

Module 1.4 - Tuples and Sets

Section A - Short Answer (2 marks)

Q59. What is a tuple? How is it different from a list?

Ans: A tuple is an ordered immutable collection in ( ). Lists are mutable and use [ ]. Tuples are faster, use less memory and can be dictionary keys.

Q60. How do you create a tuple with a single element?

Ans: Put a trailing comma: t = (5,). Without the comma, (5) is just an integer.

Q61. Which methods are available for tuples?

Ans: Only count(x) and index(x), because tuples cannot be modified.

Q62. How can a tuple be converted into a list and back? Why would you do it?

Ans: list(t) converts to a list and tuple(L) converts back. It is done to 'modify' a tuple: convert to list, change, convert back.

Q63. Explain tuple packing and unpacking.

Ans: Packing: t = 1, 2, 3 stores values in a tuple. Unpacking: a, b, c = t assigns each element to a variable; the number of variables must match.

Q64. What is the zip() function?

Ans: zip(a, b, ...) combines corresponding elements of iterables into tuples; it stops at the shortest iterable and returns an iterator. list(zip([1,2],['a','b'])) = [(1,'a'), (2,'b')].

Q65. What is a set? List its properties.

Ans: An unordered collection of unique, hashable elements in { } or set(). Properties: no duplicates, no indexing/slicing, mutable, supports mathematical operations.

Q66. Why does {} not create an empty set? How do you create one?

Ans: {} creates an empty dictionary. Use set() to create an empty set.

Q67. Differentiate between remove() and discard() in sets.

Ans: Both remove an element. If the element is missing, remove() raises KeyError while discard() does nothing.

Q68. What is a frozenset? Give one use.

Ans: An immutable version of set created with frozenset(iterable). Because it is hashable it can be used as a dictionary key or as an element of another set.

Q69. What is the difference between union() and update() in sets?

Ans: union() returns a new set containing all elements of both sets; update() adds elements into the existing set in place and returns None.

Q70. Differentiate between difference() and symmetric_difference().

Ans: A.difference(B) = elements in A but not in B. A.symmetric_difference(B) = elements in either A or B but not in both.

Section B - Descriptive (4-5 marks)

Q71. Explain the relation between tuples and lists. Give a comparison table.

Ans: Both are ordered sequences that allow duplicates, mixed types, indexing and slicing. Differences: list is mutable ([ ], many methods, slower); tuple is immutable (( ), only count/index, faster, hashable if elements are hashable). They can be converted using list() and tuple().

Q72. Explain the relation between tuples and dictionaries with examples.

Ans: items() returns key-value pairs as tuples; a list of 2-item tuples can be turned into a dictionary with dict(); tuples can be dictionary keys because they are immutable; zip() of two sequences can be converted to a dictionary.

```
d = {'a': 1, 'b': 2}
print(list(d.items()))              # [('a',1),('b',2)]
print(dict([('x', 10), ('y', 20)])) # {'x':10,'y':20}
print(dict(zip('ab', [1, 2])))      # {'a':1,'b':2}
```

Q73. Explain basic tuple operations, indexing and slicing with examples.

Ans: Concatenation (+), repetition (*), membership (in), len(), indexing t[i], slicing t[a:b:c], unpacking. Tuples cannot be modified.

```
t = (1, 2, 3, 4)
print(t + (5,))   # (1,2,3,4,5)
print(t[1:3])     # (2, 3)
print(t[::-1])    # (4,3,2,1)
```

Q74. Explain the zip() function with examples, including unzip and dict creation.

Ans: zip pairs elements positionally. Use list(zip(a,b)), dict(zip(keys, values)), and zip(*pairs) to unzip.

```
a = [1, 2, 3]; b = ['x', 'y']
print(list(zip(a, b)))     # [(1,'x'),(2,'y')]
x, y = zip(*[(1,'p'),(2,'q')])
print(x, y)                # (1, 2) ('p', 'q')
```

Q75. Explain set methods and set operations with examples.

Ans: add, update, remove, discard, pop, clear, copy; union (|), intersection (&), difference (-), symmetric_difference (^), issubset, issuperset, isdisjoint.

```
A = {1, 2, 3}; B = {2, 3, 4}
print(A | B, A & B)    # {1,2,3,4} {2,3}
print(A - B, A ^ B)    # {1} {1,4}
A.add(10); A.discard(99)
```

Q76. How can a set be traversed? Why is the order not fixed?

Ans: Use a for loop: for x in s. Sets are stored using hashing, so there is no index or guaranteed order. Use sorted(s) to traverse in sorted order.

Q77. Differentiate between set and frozenset. Explain frozenset with an example.

Ans: set is mutable (add/remove allowed) and unhashable; frozenset is immutable and hashable. frozenset supports union, intersection, difference, etc. but not add/remove/update.

```
fs = frozenset([1, 2, 2, 3])
print(fs)           # frozenset({1, 2, 3})
d = {fs: 'group'}   # allowed as dict key
```

Q78. Compare list, tuple, set and dictionary.

Ans: List [ ]: ordered, mutable, duplicates allowed. Tuple ( ): ordered, immutable, duplicates allowed. Set { }: unordered, mutable, unique elements. Dictionary {k:v}: key-value pairs, keys unique, mutable, insertion ordered.

Section C - Output / Reasoning Questions (Short Answers)

Predict the output and give a one-line reason.

Q79. print(type(5/2), 5//2, 5 % 2)

Ans: <class 'float'> 2 1 (/ gives float, // floor, % remainder).

Q80. print('Python'[1:4], 'Python'[-2:])

Ans: yth on

Q81. L = [1, 2, 3]; M = L; M.append(4); print(L)

Ans: [1, 2, 3, 4] - M and L refer to the same list; use L.copy() for a separate copy.

Q82. t = (1, 2, 3); t[0] = 9

Ans: TypeError - tuples are immutable.

Q83. print(len({1, 2, 2, 3, 3}))

Ans: 3 - duplicates are removed in a set.

Q84. d = {'a': 1}; print(d.get('b', 'NA'))

Ans: NA - get() returns the default when the key is missing.

Q85. print(list(zip([1, 2, 3], 'ab')))

Ans: [(1, 'a'), (2, 'b')] - zip stops at the shortest.

Q86. x = [3, 1, 2]; y = x.sort(); print(y)

Ans: None - sort() works in place and returns None.